

```

Start
// 1 ■■■ ■■■■■ ■■■■■■■■■■
Declare array students[]
Declare array grades[]
Declare integer n
Declare string target
Declare integer index
// 2 ■■■ ■■■■■ ■■■ ■■■■■■
Print "Enter number of students:"
Input n
// 3 ■■■ ■■■■■ ■■■■■ ■■■■■■■ ■■■■■■■■
For i = 1 to n
Print "Enter student name:"
Input students[i]
Print "Enter student grade:"
Input grades[i]
EndFor
// 4 ■■■ ■■■■■ ■■■■■■■■■ ■■ ■■■■■■ ■■■ ■■■■■■ ■■■■■■■■■ Merge Sort
Print "Sorting grades in descending order using Merge Sort..."
Call MergeSort(grades, students, 0, n-1)
// 5 ■■■ ■■■ ■■■■■■■ ■■■ ■■■■■■■■
Print "Sorted list of students and grades:"
For i = 0 to n-1
Print students[i], ":", grades[i]
EndFor
// 6 ■■■ ■■■■■ ■■ ■■■ ■■■■■■
Print "Enter student name to search for:"
Input target
index = BinarySearchByName(students, grades, target, 0, n-1)
// 7 ■■■ ■■■ ■■■■■■■■
If index != -1 then
Print "Student found:", students[index], "with grade", grades[index]
Else
Print "Student not found!"
EndIf
End
// ----- Merge Sort Algorithm -----
Procedure MergeSort(grades[], students[], left, right)
If left < right then
mid = (left + right) / 2
Call MergeSort(grades, students, left, mid)
Call MergeSort(grades, students, mid+1, right)
Call Merge(grades, students, left, mid, right)
EndIf
EndProcedure
Procedure Merge(grades[], students[], left, mid, right)
Create temporary arrays LeftGrades[], RightGrades[], LeftStudents[], RightStudents[]
Copy data into temporary arrays
Merge the two halves in descending order (highest grade first)
EndProcedure
// ----- Binary Search by Name -----
Function BinarySearchByName(students[], grades[], target, left, right)
While left <= right

```

```
mid = (left + right) / 2
If students[mid] == target then
return mid
Elseif students[mid] > target then
right = mid - 1
Else
left = mid + 1
EndIf
EndWhile
return -1
EndFunction
```